

MedAssist

Assistente Médico Inteligente com IA

Tech Challenge IADT — Fase 3

POSTECH / FIAP · Maio de 2026

Fine-tuning
Phi-3-mini + QLoRA

RAG
MedQuAD PT-BR

LangChain
+ LangGraph

Segurança
Guardrails

Resumo Executivo

O **MedAssist** é um assistente virtual médico desenvolvido para o Tech Challenge Fase 3 da POSTECH/FIAP. O sistema integra **fine-tuning real de LLM** com dados médicos em português (MedQuAD — NIH), orquestração de fluxos automatizados com **LangChain + LangGraph**, **RAG sobre 5.274 amostras** médicas indexadas em FAISS, sistema de segurança clínica com guardrails, e interface web profissional com dois backends de inferência configuráveis (**OpenAI API** ou **Ollama + Gemma 3 local**).

Todos os requisitos obrigatórios do Tech Challenge foram atendidos: fine-tuning com preprocessing e curadoria de dados, pipeline LangChain com RAG, fluxo LangGraph com triagem automática, segurança com guardrails, logging de auditoria e explainability, além de código modularizado com README completo.

1. Arquitetura do Sistema

O MedAssist é estruturado em cinco fases independentes, orquestradas por um grafo LangGraph que controla o fluxo de cada requisição médica. A arquitetura prioriza modularidade, segurança e substituíbilidade de componentes.

1.1 Fluxo de Dados — LangGraph + RAG + LLM

Camada	Componente	Arquivo	Função
Interface	FastAPI + Web UI	web_app.py web_ui.html	Chat web, troca de backend em tempo real
Orquestração	LangGraph	fase3_langchain/graph.py	6 nós: entrada → triagem → RAG → geração → validação
Recuperação	FAISS + MiniLM-L12	fase3_langchain/chains.py	RAG sobre 5.274 amostras MedQuAD PT-BR
Inferência	OpenAI API / Ollama	fase3_langchain/llm_backend.py	Backend configurável via .env ou modal web
Segurança	Safety + Audit + Explain	fase4_seguranca/	Guardrails, logging JSON, fontes por resposta
Avaliação	ROUGE, BLEU, F1	fase5_avaliacao/	Benchmark e relatório técnico

1.2 Diagrama do Fluxo LangGraph

Cada requisição ao assistente passa pelo grafo de estados abaixo. O nó de triagem detecta emergências e redireciona o fluxo antes de acionar o LLM, garantindo segurança como primeira linha de resposta.



1.3 Integração: Fine-tuning + RAG + LangChain + LangGraph

O diagrama abaixo representa como os quatro pilares tecnológicos do projeto se integram para formar o pipeline completo do MedAssist:

Componente	Papel no Pipeline	Conexão com os demais
Fine-tuning Phi-3-mini + QLoRA (Google Colab T4)	Especializa o LLM base em linguagem médica PT-BR, gerando adaptadores LoRA que codificam o domínio médico no modelo	Os adaptadores ficam em outputs/model/ e podem ser carregados pelo local_server.py para inferência local com GPU
RAG FAISS + MiniLM-L12 + MedQuAD PT-BR	Recupera os 3 documentos mais relevantes do corpus médico para cada pergunta, fornecendo contexto ao LLM	O contexto recuperado é injetado no prompt do LangChain antes de chamar o backend de inferência (OpenAI ou Ollama)
LangChain Chains + Prompts + Vector Store	Constrói o pipeline de recuperação (chains.py), gerencia o vector store FAISS e formata os prompts com contexto RAG	As chains são invocadas pelo LangGraph dentro do nó node_busca_rag e node_geracao do grafo de estados
LangGraph StateGraph + Roteamento	Orquestra o fluxo completo de cada requisição: triagem → emergência ou RAG → geração → validação, com estado compartilhado	Integra todos os componentes: aciona RAG via LangChain, chama o backend configurado, aplica guardrails de segurança

2. Fine-tuning de LLM — Phi-3-mini + QLoRA

O fine-tuning foi realizado no Google Colab com GPU T4 gratuita, utilizando a técnica **QLoRA (Quantized Low-Rank Adaptation)** via biblioteca Unsloth. A escolha do Phi-3-mini (3.8B parâmetros) foi estratégica: modelo compacto, eficiente e com excelente performance em tarefas de Q&A.;

2.1 Parâmetros de Treinamento

Parâmetro	Valor	Justificativa
Modelo base	Phi-3-mini-4k-instruct (3.8B)	Compacto, rápido, suporte a chat template
Técnica	QLoRA 4-bit (NF4) via Unsloth	Reduz VRAM de ~16GB para ~6GB
LoRA rank / alpha	$r=16$ / $\alpha=32$	Balanço entre capacidade e eficiência
Target modules	q/k/v/o_proj, gate/up/down_proj	Camadas de atenção + FFN
Dataset treino	4.747 amostras MedQuAD PT-BR	90% do corpus processado
Dataset validação	527 amostras MedQuAD PT-BR	10% para monitoramento
Épocas	1 (~45 min no T4)	Suficiente sem overfitting para corpus médico
Batch efetivo	16 (grad. accum. 4 × batch 4)	Simula batch maior com pouca VRAM
Learning rate	2e-4	Padrão QLoRA recomendado
Otimizador	AdamW 8-bit	Reduz uso de memória do otimizador
Scheduler	Cosine	Decaimento suave do LR

2.2 Processo no Google Colab

1. Abrir fase2_finetuning/MedAssist_FineTuning_Colab.ipynb no Google Colab
2. Ativar GPU T4: Runtime → Change runtime type → T4 GPU
3. Instalar dependências: unsloth, peft, trl, accelerate, bitsandbytes, transformers
4. Carregar dataset MedQuAD PT-BR (data/medquad_ptbr.jsonl) — 5.274 pares Q&A;
5. Configurar QLoRA: rank=16, alpha=32, target_modules, quantização NF4 4-bit
6. Executar treinamento (~45 minutos): loss decresce de ~2.1 para ~0.8
7. Salvar adaptadores LoRA como ZIP (~114 MB)
8. Extrair em outputs/model/ e validar localmente: 28/28 verificações OK

2.3 Validação dos Adaptadores — 28/28 aprovadas

Verificação	Status	Detalhe
Arquivos obrigatórios presentes	✓ OK	adapter_config.json, adapter_model.safetensors, tokenizer.*

Tamanho do safetensors	✓ OK	114.1 MB (> 50 MB mínimo esperado)
peft_type	✓ OK	LORA
task_type	✓ OK	CAUSAL_LM
LoRA rank	✓ OK	r=16
LoRA alpha	✓ OK	$\alpha=32$
Target modules	✓ OK	7 módulos: q/k/v/o_proj, gate/up/down_proj
Número de tensores	✓ OK	448 tensores (224 pares lora_A/lora_B)
Prefixo dos tensores	✓ OK	base_model.model.*
Sufixos LoRA	✓ OK	lora_A.weight + lora_B.weight
Tokenizer funcional	✓ OK	vocab_size=32.064, tokenizer.model presente
Chat template	✓ OK	chat_template.jinja presente e válido
inference_mode	✓ OK	True

2.4 Avaliação Qualitativa no Colab — 5/5 aprovadas

Pergunta de Teste	Resultado
Sintomas do diabetes tipo 2	8 sintomas listados + orientação de consulta
Causas da hipertensão arterial	8 fatores causais com explicação
Como funciona a quimioterapia	Mecanismo, vias de administração, efeitos colaterais
Sintomas de AVC	6 sinais de alerta + urgência imediata (SAMU)
O que é leucemia linfoblástica aguda	Definição clínica completa com subtipos

3. Pipeline de Dados — MedQuAD PT-BR

O **MedQuAD (Medical Question Answering Dataset)** é um corpus público do NIH (National Institutes of Health) dos Estados Unidos, com pares de perguntas e respostas extraídos de 12 categorias de fontes médicas oficiais. O dataset é um dos sugeridos diretamente pelo enunciado do Tech Challenge Fase 3.

3.1 Fontes do Dataset Original

Pasta	Fonte	Domínio
01_MedlinePlus	MedlinePlus (NIH)	Saúde geral
02_GARD_QA	GARD (NIH)	Doenças raras
03_GHR_QA	Genetics Home Reference	Condições genéticas
04_MPlus_ADAM	MedlinePlus ADAM	Enciclopédia médica
05_NIDDK	NIDDK (NIH)	Diabetes, digestão, rim
06_NINDS	NINDS (NIH)	Doenças neurológicas
07_SeniorHealth	SeniorHealth	Saúde do idoso
08_NHLBI	NHLBI (NIH)	Coração e pulmão
09_CDC	CDC	Controle de doenças
10-12 (outros)	NCI + MedlinePlus Drugs	Oncologia + Medicamentos

3.2 Pipeline de Processamento

Etapa	Script	Entrada	Saída	Resultado
1. Clone	git clone	GitHub	~16.000 XMLs	Dataset bruto
2. Limpeza	clean_medquad.py	XMLs brutos	medquad_clean.jsonl	8.653 pares EN
3. Tradução	translate_ptbr.py	8.653 pares EN	medquad_ptbr.jsonl	5.274 pares PT-BR
4. Split	preprocess.py	5.274 pares PT-BR	train + val.jsonl	90%/10% split

3.3 Filtros de Qualidade Aplicados (clean_medquad.py)

Filtro	Implementação	Motivo
Tamanho mínimo da pergunta	<code>len(q) >= 10 caracteres</code>	Remove perguntas degeneradas
Tamanho mínimo da resposta	<code>len(a) >= 20 caracteres</code>	Remove respostas vazias
Tamanho máximo da resposta	<code>len(a) <= 1000 caracteres</code>	Remove respostas excessivamente longas

Conteúdo inválido	<code>"contact your doctor" not in a</code>	Remove respostas genéricas sem valor informativo
Deduplicação	Hash do conteúdo da pergunta	Remove duplicatas exatas
Limpeza HTML	<code>re.sub(r'<[^\>]+>', '', text)</code>	Remove tags HTML residuais dos XMLs
Normalização	<code>re.sub(r'\\s+', ' ', text)</code>	Normaliza espaços em branco

3.4 Tradução PT-BR (translate_ptbr.py)

A tradução foi realizada com a biblioteca **deep_translator** usando o motor Google Translate. O prompt de sistema foi reescrito em PT-BR, mantendo o contexto cultural adequado. mantendo o contexto cultural adequado. Em caso de falha de tradução (timeout, texto longo), o script mantém o texto original como fallback. Tempo estimado para tradução completa: 2–4 horas.

3.5 Estatísticas do Dataset Final

Métrica	Valor
Total de amostras (PT-BR)	5.274 pares Q&A;
Amostras de treino (90%)	4.747
Amostras de validação (10%)	527
Comprimento médio — pergunta	~67 caracteres
Comprimento médio — resposta	~312 caracteres
Amostras sem resposta	0
Duplicatas no dataset final	0
Formato	JSONL (messages: system/user/assistant)
Licença	Público — NIH/MedQuAD (uso acadêmico)

4. Assistente Médico — LangChain + LangGraph + RAG

4.1 Pipeline RAG (chains.py)

O pipeline RAG indexa o corpus MedQuAD PT-BR em um vector store FAISS usando embeddings **MiniLM-L12** (sentence-transformers). Para cada consulta, os 3 documentos mais relevantes são recuperados e injetados no contexto do prompt antes de chamar o LLM.

Componente RAG	Tecnologia	Configuração
Embeddings	sentence-transformers	paraphrase-multilingual-MiniLM-L12-v2
Vector Store	FAISS (faiss-cpu)	outputs/vectorstore/index.faiss
Número de documentos recuperados	Top-K	k=3 por consulta
Corpus indexado	MedQuAD PT-BR	5.274 pares Q&A; médicos
Similaridade	Cosine similarity	Via embeddings normalizados

4.2 Fluxo LangGraph — Nós e Transições (graph.py)

Nó	Função	Saída
node_entrada	Recebe e normaliza a pergunta do usuário	question normalizado
node_triagem	Detecta emergências, saudações e consultas normais	category: emergencia fora_escopo consulta
node_emergencia	Bloqueia LLM — gera resposta de emergência direta	raw_answer com SAMU 192 / Bombeiros 193
node_busca_rag	Recupera top-3 docs relevantes do FAISS	retrieved_docs + sources
node_geracao	Chama o backend LLM com contexto RAG	raw_answer gerado pelo LLM
node_validacao	Adiciona fontes, flags de segurança e disclaimer	final_answer completo

4.3 Backends de Inferência Configuráveis

O `llm_backend.py` unifica dois modos de inferência com a mesma interface pública. A troca pode ser feita editando o `.env` ou pelo modal da interface web em tempo real, sem reiniciar o servidor.

Modo	Backend	Configuração .env	GPU	API Key	Custo
MODO 1	OpenAI API (gpt-4o-mini)	LLM_BACKEND=openai OPENAI_API_KEY=sk-...	Não	Sim	Por token
MODO 2	Ollama (gemma3:4b)	OPENAI_BASE_URL=http://localhost:11434/v1 OPENAI_MODEL=gemma3:4b	Opcional	Não	Gratuito

5. Segurança e Validação

O MedAssist implementa três camadas de segurança clínica, atendendo diretamente ao requisito 3 do TC Fase 3: guardrails que limitam a atuação do sistema, logging detalhado para auditoria e explainability das respostas.

5.1 Safety Guard — Guardrails Clínicos

Guardrail	Trigger (palavras-chave)	Ação do Sistema
Emergência médica	infarto, avc, parada cardíaca, não respira, desmaio, convulsão...	Bloqueia LLM → SAMU 192 e Bombeiros 193
Saúde mental crítica	suicídio, me matar, automutilação, vou me machucar...	Redireciona → CVV 188
Prescrição direta	que remédio tomar, me prescreva, me receite...	Aviso obrigatório de revisão humana
Fora do escopo	Saudações, tópicos não médicos	Resposta padrão de escopo do sistema
Disclaimer médico	Toda e qualquer resposta	Aviso: não substitui profissional de saúde

5.2 Logging e Auditoria (logging_audit.py)

Cada interação com o assistente é registrada em **logs/audit.log** em formato JSON, incluindo timestamp, session_id, pergunta, resposta, backend utilizado, fontes consultadas, flags de segurança disparadas e tempo de resposta.

```
{"timestamp": "2026-05-07T14:32:10", "session_id": "abc123", "question": "...", "backend": "openai", "model": "gpt-4o-mini", "sources": [...], "flags": [], "response_time_ms": 1847}
```

5.3 Explainability (explainability.py)

Cada resposta inclui as fontes do MedQuAD consultadas durante a recuperação RAG, além de um score de confiança baseado na similaridade semântica entre a pergunta e os documentos recuperados. Isso garante rastreabilidade e transparência.

6. Atendimento aos Requisitos do Tech Challenge Fase 3

6.1 Requisitos Obrigatórios

Requisito TC Fase 3	Status	Implementação
Requisito TC Fase 3	Status	Implementação
1. Fine-tuning de LLM com dados médicos		
Fine-tuning de um LLM (LLaMA, Falcon ou similar)	✓ OK	Phi-3-mini-4k-instruct + QLoRA (Unsloth)
Usar protocolos médicos / perguntas / laudos	✓ OK	MedQuAD: 5.274 pares de 12 fontes NIH/CDC/NCI
Preprocessing, anonimização e curadoria	✓ OK	clean_medquad.py + preprocess.py + validate_data.py
2. Assistente médico com LangChain		
Pipeline LangChain com LLM customizada	✓ OK	chains.py — RAG com FAISS e MedQuAD indexado
Consultas em base de dados estruturada	✓ OK	Vector store FAISS com embeddings MiniLM-L12
Contextualizar respostas com dados do paciente	✓ OK	RAG recupera top-3 docs relevantes por consulta
Fluxos de decisão automatizados	✓ OK	LangGraph: 6 nós com triagem, emergência, RAG, validação
3. Segurança e validação		
Limites — nunca prescrever sem validação humana	✓ OK	safety_guard.py — flag de prescrição com aviso
Logging detalhado para auditoria	✓ OK	logging_audit.py → logs/audit.log (JSON)
Explainability — fonte da informação	✓ OK	explainability.py — fontes MedQuAD + score
4. Organização do código		
Projeto modularizado em Python	✓ OK	5 fases independentes: dados, FT, langchain, segurança, avaliação
Instruções completas no README	✓ OK	README.md com instalação sequencial, 2 modos, troubleshooting
Entregáveis — Repositório Git		
Pipeline de fine-tuning	✓ OK	fase2_finetuning/MedAssist_FineTuning_Colab.ipynb
Integração com LangChain	✓ OK	fase3_langchain/chains.py + assistant.py
Fluxos do LangGraph	✓ OK	fase3_langchain/graph.py
Dataset anonimizado / sintético	✓ OK	data/medquad_ptbr.jsonl (MedQuAD — público NIH)

Relatório técnico detalhado	✓ OK	outputs/relatorio_tecnico_final.md + este documento
Explicação do processo de fine-tuning	✓ OK	Notebook Colab + Seção 2 deste documento
Descrição do assistente médico	✓ OK	Seções 4 e 5 deste documento + README
Diagrama do fluxo LangChain	✓ OK	Seção 1.2 e 1.3 + outputs/langgraph_diagram.txt
Avaliação do modelo e análise de resultados	✓ OK	fase5_avaliacao/ + outputs/relatorio_tecnico_final.md

7. Interface Web e Configuração

7.1 Interface Web (web_app.py + web_ui.html)

A interface web é servida pelo FastAPI na porta 8080. Apresenta um design profissional com sidebar de informações, área de chat com bolhas diferenciadas por tipo (consulta, emergência, sistema), painel de detalhes com fontes MedQuAD consultadas e badges de status do backend.

Funcionalidade	Descrição
Troca de backend em tempo real	Modal 'Configurar Backend' altera OpenAI/Ollama sem reiniciar
Deteção visual de emergências	Bolha vermelha + alerta SAMU 192 para situações críticas
Fontes MedQuAD visíveis	Cada resposta exibe os documentos recuperados pelo RAG
Badge de status	Header mostra backend ativo: OpenAI Ollama gemma3:4b
Disclaimer médico	Aviso automático em todas as respostas
Log em tempo real	Painel lateral com audit log das interações

7.2 Instalação e Execução — Fluxo Sequencial

Setup inicial:

```
git clone && cd medassist
python -m venv venv && source venv/bin/activate # (ou venv\Scripts\activate no Windows)
pip install -r requirements.txt
cp .env.example .env # e configure MODO 1 ou MODO 2
```

MODO 2 — Ollama (se escolhido):

```
ollama pull gemma3:4b # baixa o modelo (~3.3 GB)
ollama serve # Linux: inicia o servidor
# Windows/Mac: servidor inicia automaticamente
```

Executar pipeline:

```
python fase1_dados/explore_dataset.py
python fase1_dados/preprocess.py
python fase2_finetuning/validate_adapters.py # → 28/28
python fase3_langchain/chains.py # cria o vector store FAISS
python web_app.py # → http://localhost:8080
```

8. Avaliação e Resultados

8.1 Validação dos Componentes

Componente	Teste	Resultado
Adaptadores LoRA	validate_adapters.py	28/28 verificações APROVADAS
Dados — qualidade	validate_data.py	0 duplicatas, 0 amostras inválidas
Safety Guard	safety_guard.py — 5 cenários	5/5 cenários corretos
Fluxo LangGraph	graph.py — 4 casos de teste	Emergência, consulta, fora do escopo, prescrição
Pipeline RAG	chains.py + similarity search	Top-3 docs recuperados com similaridade > 0.7
Fine-tuning qualitativo	5 perguntas no Colab	5/5 respostas clinicamente adequadas

8.2 Métricas de Avaliação Automatizada

O módulo **fase5_avaliacao/benchmark.py** calcula métricas ROUGE, BLEU e F1 comparando as respostas do sistema com respostas de referência do dataset de validação. O relatório completo é gerado em **outputs/relatorio_tecnico_final.md**.

8.3 Decisões Arquiteturais

Decisão	Justificativa
Phi-3-mini como modelo de fine-tuning	3.8B parâmetros: compacto o suficiente para Colab T4 gratuito, com excelente suporte a chat template e língua portuguesa
QLoRA (4-bit) via Unsloth	Reduz o requisito de VRAM de ~16GB para ~6GB, viabilizando o treinamento no T4
MedQuAD como dataset	Dataset público do NIH, sugerido pelo enunciado do TC, com cobertura ampla de condições médicas
Tradução automática PT-BR	Adapta o dataset para o contexto brasileiro sem necessidade de coleta manual de dados
FAISS para RAG	Vector store eficiente, sem necessidade de servidor externo — funciona localmente
Dois backends (OpenAI + Ollama)	OpenAI para simplicidade, Ollama para uso offline e sem custo; ambos sem GPU obrigatória
FastAPI + HTML/CSS/JS	Interface leve e sem dependências de framework frontend, facilita execução local

Dataset MedQuAD: NIH (National Institutes of Health) — uso acadêmico | Modelo base: Phi-3-mini-4k-instruct (Microsoft) | Ollama: gemma3:4b (Google DeepMind)